



# OASIS Security Report

## Insufficient Input Validation Security Analysis

**Date:** 2026-06-11 10:19:12 — **Model:** mistral:latest

**Vulnerability:** Insufficient Input Validation

**Project:** clean\_contracts

**Analysis root:** `../clean`

### Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

## Summary

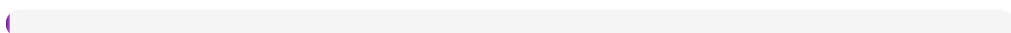
Analyzed **2** file(s) for Insufficient Input Validation.

[Return to table of contents](#)

## Statistics

**Files analyzed:** 2    **Total findings:** 2

Critical



0

|        |             |   |
|--------|-------------|---|
| High   | <div></div> | 0 |
| Medium | <div></div> | 2 |
| Low    | <div></div> | 0 |

[Return to table of contents](#)

## Files

| File                                                                   | Similarity |
|------------------------------------------------------------------------|------------|
| <code>/work/project/dataset/clean/modifier_reentrancy_fixed.sol</code> | 0.602      |
| <code>/work/project/dataset/clean/simple_dao_fixed.sol</code>          | 0.583      |

[Return to table of contents](#)

## Detailed analysis

### File 1: /work/project/dataset/clean/modifier\_reentrancy\_fixed.sol

Similarity score: 0.602

Source lines: 1-42

#### ► Finding 1.1: Vulnerability found (Medium)

##### Vulnerable code

```
airDrop() function
```

##### Explanation

The airDrop function checks for hasNoBalance before supportsToken, making it vulnerable to a reentrancy attack by allowing a contract with any balance to call the function.

##### Impact

A malicious actor could potentially drain all available tokens.

##### Entry point

airDrop() function

##### Execution path

```
```  
1. Contract calls airDrop()  
2. supportsToken is checked (safe)  
3. hasNoBalance is checked (vulnerable)  
4. Token balance updated (unsafe)  
5. Contract re-enters step 1 by exploiting reentrancy vulnerability````
```

##### Parameters

None

##### Exploitation steps

- Call airDrop function
- Exploit reentrancy attack

##### Conditions

Attacker must control a contract supporting the Nu Token.

## Remediation

Ensure that hasNoBalance check is executed before supportsToken in the airDrop function.

### Secure example

```
```\nsupportsToken() and hasNoBalance modifiers swapped in the airDrop function:\nfunction airDrop() supportsHasNoBalance public{\n  if(supportsToken()) {\n    tokenBalance[msg.sender] += 20;\n  }\n}\n```\n
```

**File 2: /work/project/dataset/clean/simple\_dao\_fixed.sol**

**Similarity score:** 0.583

**Source lines:** 1-34

► **Finding 2.1: Vulnerability found (Medium)**

**Vulnerable code**

```
donate(address to) payable public{
    credit[to] += msg.value;
    },
```

**Explanation**

The `donate` function does not validate the input `to` address, making it vulnerable to arbitrary transfers.

**Impact**

An attacker could drain the contract's funds by repeatedly donating to an attacker-controlled address.

**Entry point**

donate function

**Execution path**



**Parameters**

to

**Exploitation steps**

- Send a transaction to the contract with a malicious `to` address
- Monitor contract's balance for drained funds

**Conditions**

Attacker has an Ethereum account and is capable of interacting with smart contracts

## Remediation

Implement input validation on the `to` parameter before crediting it.

## Secure example

```
function donate(address safeTo) payable public{
    require(SafeMath.requireAddress(safeTo), "Invalid address provided.");
    credit[safeTo] += msg.value;
},
```

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)